COMPUTER SCIENCE + COMPUTER ENGINEERING FOR GAMES

PACKAGE 07 / VOLUME 06

Start Package 07

Your route through Volume 06

UPPER UNDERGRADUATE
Modules 26-30 · Lessons 26.1-30.4 · Gate F

Edition: September 28, 2026

Original expanded instruction, worked examples, quizzes, implementation contracts and evidence-based review. The learner performs the projects and platform/human validation. This self-study curriculum is not an accredited degree.

Contents

Start Package 07 **3**

Volume 06: Game Systems and Production 3

Readiness and placement 3

Open the package 4

First working session 4

Native and integration work 5

How to study and assess 5

What you should have at the end 6

Start Package 07

Volume 06: Game Systems and Production

This bundle expands the sixth volume of the established Computer Science and Computer Engineering for Games program. It preserves Modules 26-30, Lessons 26.1-30.4 and Checkpoint F. Package numbers and volume numbers are different: Package 07 is not Volume 07. The next volume is Computer Engineering and Digital Design, Modules 31-35.

The package contains 20 expanded lessons, 20 full laboratory specifications, 20 runnable Python demonstrations, 20 exercise/reference pairs, 100 lesson questions and 25 additional module-review questions. The original 60 lesson questions are retained verbatim, with 40 new application questions. Answers are separated in ANSWER_KEY and hidden behind a reveal control in the browser course. Five module projects culminate in Gate F; they are not six unrelated full-sized projects.

The original roadmap assigns 420-650 planning hours to Volume 6, including study, implementation, integration, review and the capstone. These are author estimates, not reading time, credit hours or a guaranteed schedule. A brief kernel exercise does not replace the full laboratory, and a successful package self-test does not certify a student’s engineering competence.

Readiness and placement

The normal route is through Volume 5 and Gate E: a small engine with explicit ownership, fixed-step simulation, rendering or a software path, asset handling, diagnostics and reproducible builds. Module 30 also retains the original Gate D software-engineering evidence requirement. Review the exact module prerequisites in the handbook rather than skipping mathematics or systems topics that happen not to appear in a short demo.

Before Lesson 26.1, independently demonstrate these readiness checks: explain local versus world coordinates; compute a dot product and a 2D cross product; distinguish velocity from acceleration; build and test a small C++ program; describe a race and a resource lifetime; and rerun a fixed input tape in your engine. Repair any gap with the corresponding prior-volume lessons and record the repair in templates/PREREQUISITE_REPAIR.md. These checks are a diagnostic, not a validated placement examination.

You may study the mathematical references before your engine is fully ready, but engine-transfer work and Gate F remain blocked until its prerequisites work. A new project written entirely in an off-the-shelf engine does not substitute for the central engine-building objective. Existing engines and maintained libraries are excellent comparison targets.

Open the package

Extract the entire ZIP and preserve its directory layout. Open STUDENT_HANDBOOK.pdf for teaching, LAB_WORKBOOK.pdf for contracts and evidence, and CAPSTONE_SPEC.pdf for the final game. SYSTEMS_REFERENCE.pdf summarizes units, formulas and boundary conventions. NATIVE_INTEGRATION_GUIDE.pdf explains what must move from Python teaching kernels into native/platform engine code.

COURSE.html is the offline searchable course. It contains all lessons, quizzes, hidden answers, module projects, full lab cards, module reviews and references. It uses no remote scripts, account or server. Progress is saved locally when the browser permits storage; file-URL storage policies vary. Export progress regularly with the course's JSON export control. The progress file is a learner record, not an automatic approval.

The source/ folder contains editable Markdown and structured JSON. modules/ contains five smaller PDFs. templates/ contains empty evidence forms, not completed student records. evidence/ records automated checks performed on this package's supplied implementations, clearly separated from the manual/platform work still required of the learner.

First working session

Use a terminal in the extracted Game_Engineering_Package_07 folder. Check the interpreter and run one demonstration. On systems where the Python command is named python3, substitute python3; on Windows the installed launcher may be py -3.

```
python --version
python starter/python/demos/lesson_26_1.py
python starter/python/run_tests.py --target references
python starter/python/test_properties.py
```

Python 3.11+ is the chosen syntax baseline. Actual automated execution was on Python 3.13.5 on Linux; other Python/OS combinations are not represented as separately tested. No Python package install, GPU, cloud account or purchased dataset is required for these kernels and companion simulations.

Read Lesson 26.1, predict its worked trace, and implement starter/python/exercises/lesson_26_1.py. Each starter has a solve(payload) contract and intentionally raises NotImplementedError until you replace it. Reference code is for comparison after an independent attempt; importing it into your solution defeats the exercise.

```
python starter/python/run_tests.py --target exercises --lesson 26.1
```

A failing unimplemented starter is expected. It is not silently counted as passing or skipped. The tests cover the documented teaching contract and selected invalid inputs; they are not exhaustive security certification. Complete the full lab card, then the engine transfer, before marking the lab passed.

Native and integration work

C++17 and CMake are used by the native offline audio harness. This harness opens no audio device. It illustrates bounded voice and command work, prepared storage and a single-producer/single-consumer queue. Its passing tests do not prove thread stress, device playback, absence of underruns or a hard real-time deadline.

```
cmake -S starter/cpp -B build/audio
cmake --build build/audio --config Release
ctest --test-dir build/audio -C Release --output-on-failure
```

The five Python companions run independently and print structured results:

```
python starter/integration/physics_lab.py
python starter/integration/learning_lab.py
python starter/integration/network_lab.py
python starter/integration/service_lab.py
python starter/integration/release_lab.py
```

Physics includes analytic convergence, positional residuals and an offset impulse; it is not a maintained-library comparison. Learning uses synthetic independent episode seeds. Networking simulates two clients in one process with seeded transport faults and a labeled reliable drain. Services use SQLite, failure injection and a fresh interpreter read. Release tests use temporary folders and injected exceptions, not power-cut testing.

For the full labs, add a maintained physics-library comparison, an actual platform audio callback/device test, and a local process/socket transport adapter. No paid cloud service is required. When a required device or platform is unavailable, record the work as deferred rather than passed. Gate F must not claim that deferred feature’s validation.

How to study and assess

For each lesson, read the goal and worked examples, predict before execution, complete assigned reading, implement the kernel, perform the full lab and transfer, then answer all five questions without the key. Record what failed and why. Use the reference solution to diagnose a misconception, then solve a fresh variation without copying it.

Each lesson question is worth 0-2 formative points: 0 for incorrect/missing, 1 for a partially correct explanation, and 2 for correct reasoning including the requested calculation or condition. The expanded quiz is out of 10, not the original primer’s six-point scale. Use 8/10 as a review target, but do not pass a lesson with a central misconception or an incomplete laboratory. Redo the missed concept and a changed input before continuing.

The five module-review questions are additional transfer checks, also scored out of 10 when useful. They do not replace the module project review. Gate F has a separate 100-point rubric plus mandatory blockers; a high numerical score cannot compensate for broken saves, an unlaunchable release or missing independent review.

What you should have at the end

A small original game on your engine, an evidence-backed physics subsystem, articulated/animated motion, bounded audio with a critical visual alternative, inspectable AI, local multiplayer demonstration, settings/save recovery and a reproducible package. The target is a coherent and explainable small system, not commercial art, a large map or a revenue milestone.

Keep the final review honest: supplied example outputs belong to this package; your capstone measurements belong to your own engine, build and machine. A self-study curriculum does not award an accredited degree or substitute for professional assessment.